

Understanding Entropy in FRET Requirements

Three separate ideas, calculated step by step

Worked system: a Boolean request–response component.
Examples: `when request` and `whenever request`, each with a three-tick deadline.
Analysis baseline: `FRET_toy_project`, commit 6443ce8.
Prepared: 27 September 2026.

Key idea. An entropy number is meaningful only after specifying what varies, which outcomes are possible, and how probabilities are assigned. The three experiments use different outcomes and must be interpreted separately.

Experiment	Question answered
Meaning entropy	Which candidate meaning is intended?
Behavioral entropy	How many finite behaviors remain permitted?
Clarification gain	How much uncertainty should the next question remove?

Learning outcomes. After these notes, you should be able to calculate Shannon entropy by hand, distinguish spellings from meanings, enumerate the setup example’s permitted traces, update interpretation probabilities after an answer, and reproduce the implemented outputs.

Prerequisites. Basic probability, Boolean logic, and logarithms. All temporal concepts used in the examples are introduced here.

Scope. These are companion analyses using real FRET-generated formulas. They do not add a native entropy panel to NASA FRET. Experimental candidate meanings and probabilities are analyst assumptions; no stakeholder study, unbounded proof, or engineering acceptance is claimed.

1. Read the requirements before calculating

The two statements come from the setup tutorial export. They are already precise FRETish requirements; the experiments do not establish that these statements are ambiguous.

REQ-001

when request the ResponseSystem shall within 3 ticks satisfy response

REQ-002

whenever request the ResponseSystem shall within 3 ticks satisfy response

Term	Meaning in this experiment
request	Boolean input: false (0) or true (1).
response	Boolean proposition: false (0) or true (1).
when	Trigger when request changes from false to true; an initially true request also triggers.
whenever	Trigger at every tick at which request is true.
within 3 ticks	Response may be true at the trigger tick or at one of the next three ticks.
satisfy	The response proposition must hold as specified by the timing clause.

Let r_t and s_t denote request and response at tick t . The trigger predicates are

$$T_{\text{when}}(t) = r_t \wedge (t = 0 \vee \neg r_{t-1}),$$

$$T_{\text{whenever}}(t) = r_t.$$

The first expression is interpreted piecewise: at tick zero there is no previous sample.

A distinguishing execution.

Tick	0	1	2	3	4
Request	1	1	1	1	1
Response	1	0	0	0	0

REQ-001 triggers only at tick 0 and is satisfied immediately. REQ-002 also triggers at tick 1; there is no response at ticks 1–4, so that obligation fails. One response need not satisfy every later trigger.

Key idea. A tick is a discrete sample, not an assumed second. No physical duration is specified in this setup.

2. Understand finite traces and FRET's boundary rule

A *trace* is a sequence of signal values. At horizon h , the trace has ticks $0, \dots, h-1$. Horizon six means six samples, not six intervals after an initial sample.

For a trigger at tick t with deadline d , the implemented finite semantics require

$$\underbrace{\exists j \in [t, \min(t+d, h-1)] : s_j = 1}_{\text{a response is observed}} \quad \vee \quad \underbrace{h-1 < t+d}_{\text{the trace ends before the deadline}} .$$

This is FRET's weak treatment of an unfinished deadline. If the deadline tick itself is present, a missing response fails. The boundary rule is part of the formal semantics, not missing data silently treated as success by our analyzer.

t	d	Last tick	No response observed: result
0	3	2	Permitted: deadline has not expired.
0	3	3	Fails: the full deadline window is present.
3	3	5	Permitted: tick 6 is beyond the trace.
3	2	5	Fails: deadline expires at tick 5.

At horizon six and deadline three, only triggers at ticks 0, 1, 2 can fail. Triggers at ticks 3, 4, 5 cannot yet fail under this finite convention.

Reading the generated formulas. The implementation evaluates FRET's `ftExpanded` output. The subset needed here includes

Symbol	Interpretation
$\neg, \wedge, \vee, \rightarrow$	Not, and, or, implies.
$X\phi$	Strong next: ϕ holds at the next tick; false if no next tick exists.
$F_{[a,b]}\phi$	ϕ holds at some available tick between offsets a and b .
LAST	True only at the last tick.
$\phi V \psi$	Release: ψ persists until and including a point satisfying ϕ , or through the end if that point never occurs.

Key idea. Changing the horizon or replacing weak unfinished deadlines with another convention changes the experiment. Always report the boundary rule beside a behavioral count.

3. Shannon entropy from the beginning

Let a discrete random variable Z have possible outcomes $z_1, \dots, z_K$, with probabilities $p_1, \dots, p_K$. Require

$$p_i \geq 0, \quad \sum_{i=1}^K p_i = 1.$$

The information associated with observing outcome z_i is its *surprisal*, $-\log_2 p_i$. Shannon entropy is the expected surprisal:

$$H(Z) = - \sum_{i:p_i>0} p_i \log_2 p_i.$$

The base-two logarithm gives units of bits. By convention, a zero-probability outcome contributes zero; the program skips that term instead of evaluating $\log_2 0$.

Example A: two equally likely meanings.

$$H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = \frac{1}{2} + \frac{1}{2} = 1 \text{ bits.}$$

Example B: a confident but non-certain distribution.

$$H(0.9, 0.1) = -0.9 \log_2(0.9) - 0.1 \log_2(0.1) \approx 0.468996 \text{ bits.}$$

Example C: a single remaining outcome.

$$H(1, 0) = -1 \log_2 1 = 0 \text{ bits.}$$

Zero says the distribution has one supported outcome. It does not show that the outcome is correct, that an omitted alternative is impossible, or that the system passed its tests.

Uniform shortcut. If each of K outcomes has probability $1/K$,

$$H = -K \frac{1}{K} \log_2 \frac{1}{K} = \log_2 K.$$

Thus 4 equally likely outcomes give 2 bits, and 4,096 give 12 bits. For a fixed K , a uniform distribution has the maximum entropy $\log_2 K$.

Weights versus probabilities. The interactive meaning page accepts nonnegative weights w_i , then uses $p_i = w_i / \sum_j w_j$. For weights (2, 1, 1), probabilities are (0.5, 0.25, 0.25). An all-zero vector is invalid. The Python analysis inputs require probabilities that already sum to one.

Key idea. Entropy measures a specified distribution. It cannot supply its own probabilities or certify that the listed outcomes are complete.

4. Idea 1: group spellings into meanings

For each setup requirement, the experiment constructs the following candidates. These are research inputs, not measured interpretations supplied by NASA or a stakeholder.

Candidate	Change relative to the original	Prior
Original	Unchanged FRETish statement	0.25
Punctuation alias	Add a final period; same formal meaning	0.25
Tighter deadline	Replace 3 ticks with 2 ticks	0.25
Alternate trigger	Exchange when and whenever	0.25

Step 1: calculate entropy over candidate labels. There are four equally weighted labels, so

$$H_{\text{labels}} = -4(0.25 \log_2 0.25) = 2 \text{ bits.}$$

Step 2: compile and compare behaviors. FRET compiles every candidate. The companion evaluator calculates each formula's true/false value on every six-tick trace. Candidates with identical 4,096-entry truth vectors form a bounded meaning class.

Step 3: sum probability mass within each class. At horizon six, the original and punctuation alias coincide. The other candidates are distinguishable. Class probabilities become

$$(p_1, p_2, p_3) = (0.25 + 0.25, 0.25, 0.25) = (0.5, 0.25, 0.25).$$

Step 4: calculate meaning entropy.

$$\begin{aligned} H_{\text{meaning}} &= -0.5 \log_2 0.5 - 0.25 \log_2 0.25 - 0.25 \log_2 0.25 \\ &= 0.5 + 0.5 + 0.5 = \boxed{1.5 \text{ bits}}. \end{aligned}$$

Both setup examples have this same experimental result. If we retain only the original precise FRETish statement, its single-interpretation baseline is zero bits.

Why the grouping matters. Splitting a meaning's fixed probability mass among equivalent spellings must not increase uncertainty about meaning. Adding labels and then assigning a fresh uniform prior is a different operation; it can change class masses.

Key idea. The equivalence claim is bounded. At two ticks these candidates all accept every trace under the weak deadline convention, so they form one bounded class. That does not prove they are equivalent for longer executions.

5. Idea 2: define the behavioral universe

Here the random variable is a *finite trace*, not an intended meaning. Fix the following experimental settings before counting:

- Two Boolean signals: request and response.
- Six ticks: 0, 1, 2, 3, 4, 5.
- No restrictions on the environment or on consecutive signal values.
- FRET's finite semantics, including unfinished deadlines.
- A uniform distribution over all trace assignments.

At each tick there are four signal pairs: (0, 0), (0, 1), (1, 0), (1, 1). Therefore

$$|\Omega_6| = 4^6 = 2^{12} = 4,096, \quad H(\Omega_6) = 12 \text{ bits.}$$

For a requirement set R , let $S_6(R) \subseteq \Omega_6$ be the traces satisfying all requirements in R . Conditioning a uniform prior on membership in a nonempty set leaves a uniform distribution on that set. Hence

$$H_{\text{behavior}}(R; 6) = \log_2 |S_6(R)|.$$

The additional information supplied by requirement r is

$$\Delta I(r \mid R; 6) = \log_2 \frac{|S_6(R)|}{|S_6(R \cup \{r\})|}.$$

For these uniform sets this equals the difference in behavioral entropies. It also equals $-\log_2 P(r \mid R)$, with probability induced by the uniform trace distribution. It is not a general claim that conditioning always decreases realized entropy under arbitrary nonuniform priors.

Warm-up. Suppose a requirement leaves 256 of 1,024 permitted traces:

$$H_{\text{before}} = 10, \quad H_{\text{after}} = 8, \quad \Delta I = \log_2(1,024/256) = 2 \text{ bits.}$$

If the count is unchanged, marginal information is zero. If the remaining set is empty, conditioning is undefined: report an inconsistent result, not zero entropy. A singleton set has zero behavioral entropy, but does not prove the sole behavior is safe or intended.

Key idea. More permitted behavior is more freedom within this universe. It is not automatically lower requirement quality. These are counts of traces, not implementations or physical designs.

6. Derive REQ-001's 3,848 traces by hand

There are 12 free Boolean values before constraints: six requests and six responses. To count satisfying traces, count the failing traces and subtract them from 4,096.

Let E_t mean that the **when** obligation triggered at tick t fails. Only $t = 0, 1, 2$ can fail, because later deadlines extend beyond tick 5.

Event	Values fixed by the failure event	Free bits	Count
E_0	$r_0 = 1, s_0 = s_1 = s_2 = s_3 = 0$	7	128
E_1	$r_0 = 0, r_1 = 1, s_1 = \dots = s_4 = 0$	6	64
E_2	$r_1 = 0, r_2 = 1, s_2 = \dots = s_5 = 0$	6	64

For example, E_0 fixes five of the twelve bits, leaving $2^7 = 128$ assignments. Each E_t includes every possible setting of its unconstrained bits.

Do not simply add these counts. A trace may belong to more than one failure event. Inclusion–exclusion gives

$$\begin{aligned}
 |E_0 \cup E_1 \cup E_2| &= |E_0| + |E_1| + |E_2| \\
 &\quad - |E_0 \cap E_1| - |E_0 \cap E_2| - |E_1 \cap E_2| \\
 &\quad + |E_0 \cap E_1 \cap E_2|.
 \end{aligned}$$

Compute the intersections.

- $E_0 \cap E_1 = \emptyset$: r_0 cannot be both 1 and 0.
- $E_1 \cap E_2 = \emptyset$: r_1 cannot be both 1 and 0.
- $E_0 \cap E_2$: $r_0 = 1, r_1 = 0, r_2 = 1$, and all six responses are 0. Only r_3, r_4, r_5 remain free, giving $2^3 = 8$ traces.
- The triple intersection is empty because it contains contradictory adjacent trigger events.

Therefore

$$N_{\text{fail,when}} = 128 + 64 + 64 - 8 = 248,$$

$$N_{\text{pass,when}} = 4,096 - 248 = 3,848.$$

The implementation's exhaustive enumeration produces exactly the same count.

7. Derive REQ-002's 3,784 traces by hand

Now **whenever** triggers at every true request tick. There is no rising-edge restriction on the previous request. Let E_t mean that the **whenever** obligation at tick t fails, again for $t \in \{0, 1, 2\}$.

Event	Fixed values	Count
E_0	$r_0 = 1$; responses 0–3 are all 0	$2^7 = 128$
E_1	$r_1 = 1$; responses 1–4 are all 0	$2^7 = 128$
E_2	$r_2 = 1$; responses 2–5 are all 0	$2^7 = 128$

Step 1: add the individual failure counts. The provisional total is $3 \times 128 = 384$, but overlapping failures have been counted more than once.

Step 2: subtract each pairwise intersection.

Intersection	Fixed bits and remaining freedom	Count
$E_0 \cap E_1$	Two requests true; responses 0–4 zero. Five free bits.	32
$E_1 \cap E_2$	Two requests true; responses 1–5 zero. Five free bits.	32
$E_0 \cap E_2$	Two requests true; all six responses zero. Four free bits.	16

Step 3: restore the triple intersection. For $E_0 \cap E_1 \cap E_2$, requests 0–2 are true and all responses are zero. The three later request bits remain free, giving 8 traces.

Step 4: finish inclusion–exclusion.

$$N_{\text{fail,whenever}} = 384 - (32 + 32 + 16) + 8 = 312.$$

$$N_{\text{pass,whenever}} = 4,096 - 312 = 3,784.$$

Interpret the difference. Every rising-edge trigger is also a true-request tick. Consequently, the original **whenever** requirement is at least as restrictive as the original **when** requirement under this semantics. The 64-trace difference consists of behaviors accepted by **when** but rejected by **whenever**.

Key idea. The two counts quantify different restrictions even though the illustrative interpretation distributions in Idea 1 have the same entropy.

8. Convert trace counts into entropy and information

Insert the verified counts into the uniform-set formula:

$$\begin{aligned} H_{\text{when}} &= \log_2(3,848) = 11.909893 \text{ bits}, \\ H_{\text{whenever}} &= \log_2(3,784) = 11.885696 \text{ bits}. \end{aligned}$$

Before either requirement is imposed, entropy is 12 bits. The information supplied relative to that unrestricted universe is

$$\begin{aligned} I_{\text{when}} &= 12 - 11.909893 = 0.090107 \text{ bits}, \\ I_{\text{whenever}} &= 12 - 11.885696 = 0.114304 \text{ bits}. \end{aligned}$$

Why are these information values small? Many uniformly sampled traces satisfy a response obligation, and late triggers cannot fail before the finite trace ends. These numbers describe this particular six-tick universe. They are not a percentage of safety, completeness, or implementation correctness.

Conditional contribution: order matters. Write W for the original **when** requirement and V for the original **whenever** requirement. Since $S_6(V) \subseteq S_6(W)$,

$$|S_6(W \wedge V)| = 3,784.$$

Adding W after V gives

$$\Delta I(W | V) = \log_2 \frac{3,784}{3,784} = \boxed{0 \text{ bits}}.$$

Adding V after W gives

$$\Delta I(V | W) = \log_2 \frac{3,848}{3,784} = \boxed{0.024197 \text{ bits}}.$$

The zero contribution is redundancy relative to the existing context. It does not imply that the redundant requirement is poorly worded or lacks traceability value.

Requirement set	Admissible traces	Behavioral entropy
None	4,096	12.000000
W only	3,848	11.909893
V only	3,784	11.885696
W and V	3,784	11.885696

Key idea. Behavioral entropies of requirements cannot simply be added. Their permitted trace sets overlap, and the marginal information depends on what is already required.

9. Idea 3: ask a question with measurable value

Return to uncertainty about *meaning*. Let M be the intended meaning class and q a proposed clarification question. In this implementation, a question shows a finite trace and asks whether the intended requirement should permit it.

An answer $A \in \{\text{yes}, \text{no}\}$ partitions the candidate classes. We assume a reliable answer and equal question costs for this experiment. If class i accepts the trace, its likelihood of a yes answer is 1; otherwise it is 0.

Step 1: predict the answer probabilities.

$$P(a | q) = \sum_i P(a | M_i, q) p_i.$$

Step 2: update the candidate probabilities after that answer.

$$P(M_i | a, q) = \frac{P(a | M_i, q) p_i}{P(a | q)}.$$

This is Bayes' rule. An answer with zero predicted probability cannot be normalized under the current candidate model; it indicates that the model or candidates need revision.

Step 3: calculate expected remaining entropy.

$$\mathbb{E}[H | q] = \sum_a P(a | q) H(M | a, q).$$

Step 4: subtract it from the current entropy.

$$\boxed{\text{EIG}(q) = H(M) - \mathbb{E}[H | q].}$$

This is expected information gain, equivalently the mutual information between meaning and answer under the stated model. It is not an entropy of behaviors and not the information gain of an answer that has already been observed.

The analyzer evaluates every trace, keeps one representative per distinct informative answer partition, and ranks the partitions by EIG. After an answer, the browser reranks questions using the posterior.

Useful bounds. With reliable binary answers, EIG is at most one bit and at most the current meaning entropy. A question whose two answers each have probability one half reaches the one-bit limit if the answer is determined by the meaning class.

Key idea. A hypothetical yes/no branch is a calculation, not stakeholder evidence. The saved initial outputs contain no actual answers.

10. Worked clarification: REQ-001

Use class probabilities $(0.5, 0.25, 0.25)$ for original **when**/3, tighter **when**/2, and alternate **whenever**/3.

The highest-ranked question in the saved output uses this trace:

Tick	0	1	2	3	4	5
Request	1	1	0	1	0	0
Response	1	0	0	0	0	0

Meaning	Permits?	Explanation
Original when/3	Yes	Tick 0 responds immediately. Tick 3's deadline is tick 6, beyond the trace.
Tighter when/2	No	The tick 3 trigger has no response by its deadline at tick 5.
Alternate whenever/3	No	The tick 1 trigger has no response by tick 4.

Calculate the yes branch. Only the original meaning permits the trace:

$$P(\text{yes}) = 0.5, \quad P(M \mid \text{yes}) = (1, 0, 0), \quad H_{\text{yes}} = 0.$$

Calculate the no branch. The other meanings have total probability $0.25 + 0.25 = 0.5$:

$$P(\text{no}) = 0.5, \quad P(M \mid \text{no}) = (0, 0.5, 0.5), \quad H_{\text{no}} = 1.$$

Average over the answers before asking.

$$\mathbb{E}[H \mid q] = 0.5(0) + 0.5(1) = 0.5 \text{ bits.}$$

$$\text{EIG}(q) = 1.5 - 0.5 = 1 \text{ bits.}$$

After a yes answer the realized reduction is 1.5 bits; after a no answer it is 0.5 bits. Their expectation is one bit. The next question is chosen from the updated distribution, not from the original ranking alone.

11. Worked clarification: REQ-002

The class probabilities are again $(0.5, 0.25, 0.25)$, now for original **whenever**/3, tighter **whenever**/2, and alternate **when**/3.

One highest-ranked question uses the following trace:

Tick	0	1	2	3	4	5
Request	0	0	0	1	0	0
Response	0	0	0	0	0	0

The three-tick candidates permit it because the deadline at tick 6 is beyond the trace. The two-tick candidate rejects it because its deadline expires at tick 5.

Yes branch. Original and alternate meanings remain:

$$P(\text{yes}) = 0.5 + 0.25 = 0.75, \quad P(M \mid \text{yes}) = (2/3, 0, 1/3).$$

$$H_{\text{yes}} = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.918296 \text{ bits.}$$

No branch. Only the tighter deadline remains:

$$P(\text{no}) = 0.25, \quad P(M \mid \text{no}) = (0, 1, 0), \quad H_{\text{no}} = 0.$$

Expected reduction.

$$\mathbb{E}[H \mid q] = 0.75(0.918296) + 0.25(0) = 0.688722,$$

$$\text{EIG}(q) = 1.5 - 0.688722 = \boxed{0.811278 \text{ bits}}.$$

Why not one bit? These meanings form an implication chain:

$$\text{whenever}/2 \implies \text{whenever}/3 \implies \text{when}/3.$$

No trace can be accepted by the middle meaning and rejected by both extremes, or vice versa. The available informative partitions split prior mass into 0.25 and 0.75, rather than 0.5 and 0.5. With reliable binary answers their gain equals $H(0.25, 0.75) = 0.811278$ bits.

Key idea. Equal initial entropy does not imply equally informative available questions. The logical relationships between candidates determine which partitions can be realized.

12. Reproduce and explore the implementation

Step 1: locate the repository. Use the existing `FRET_toy_project` checkout. The two source statements are in `tutorial/FRET_Tutorial_TR.json`. All three implementations are under `entropy/`.

Step 2: regenerate FRET formulas when needed. From the repository root, with Node and the FRET dependencies installed:

```
node entropy/compile.cjs
```

This compiles eight candidate statements and records compiler/source hashes. Without Node, the checked-in compiled inputs still support the Python analyses.

Step 3: regenerate all calculations and pages. Requires Python 3.10 or later, using only its standard library:

```
python entropy/run_all.py --horizon 6
python entropy/test_entropy.py
```

The first command creates three result files, three standalone pages, separate FRET import files, and a combined page. The second checks the numerical and semantic contracts.

Step 4: open the report. Open `entropy/index.html` locally. It works offline. The existing local preview is at `http://127.0.0.1:8767/entropy/index.html` while that server is running.

Step 5: use the three separate pages.

1. **Meaning:** inspect classes; change weights to $(2, 1, 1)$, then $(1, 0, 0)$. Observe 1.5 and 0 bits. These exploratory controls do not edit FRET or the saved results.
2. **Behavior:** compare individual counts and the contribution given the other requirement. Expand “Actual FRET output” to read the generated semantics.
3. **Clarification:** inspect the trace before answering. Yes/no updates probabilities. Unsure preserves them. Neither reopens the candidate set and pauses questions. Reset starts over; Export answer ledger downloads the session as JSON.

Treat practice answers as practice, not stakeholder elicitation data. The page does not authenticate respondents or save answers automatically.

Step 6: import into FRET. Each numbered analysis directory contains `fret-project.json`. Use FRET’s import control to create the three analysis projects. Calculated snapshots appear in requirement rationales; interactive entropy controls remain in the companion page. The eight/eight/two imported records are analysis views of two source requirements, not 18 independent experimental subjects.

13. Audit the outputs and avoid common errors

Where each result comes from.

File under entropy/	Purpose
inputs/analysis.json	Candidate priors, source provenance, compiler hash.
inputs/fret-project.json	Real FRET compiler output for all eight candidates.
common.py	Finite-formula evaluator, trace enumeration, entropy and class grouping.
01_semantic_interpretation/	Meaning classes, priors, entropy, independent entry point.
02_behavioral_freedom/	Counts, entropy, conjunction and conditional contributions.
03_active_clarification/	Distinguishing traces, branch posteriors and expected gains.
validation.json	Recorded test status and comparison scope.

What was checked. Eight test methods passed. For each of eight candidates, the evaluator was compared with an independent trigger/deadline obligation implementation on every trace at horizons 1–6:

$$8 \sum_{h=1}^6 4^h = 8(5,460) = 43,680 \text{ comparisons.}$$

Other checks cover invalid probabilities, unsupported formulas, finite boundaries, aliases, bounded equivalence, inconsistency, redundancy, witnesses and posterior normalization. The reported behavioral counts also have the independent hand derivations in Lessons 6–7.

What this does not establish. It is not external model-checker agreement, unbounded equivalence, system realizability, human probability calibration, a controlled user study, or physical acceptance testing. A parser that handles these formulas is not a complete FRET or LTL implementation.

Common interpretation errors.

- “Zero meaning entropy means correct.” It may simply mean one candidate was supplied.
- “More behavior entropy means more ambiguity.” Behavioral freedom and uncertainty about meaning are different quantities.
- “1.5 bits was measured from NASA users.” The prior is explicitly illustrative.
- “A six-tick equivalence is universal.” Longer traces may distinguish the candidates.
- “The question always removes its EIG.” EIG is an expectation before observing the answer.
- “Lower entropy is enough for acceptance.” Acceptance requires separate, current verification evidence and a reviewed decision.

14. Exercises, solutions, and further reading

Exercise 1. Calculate $H(0.8, 0.1, 0.1)$. What assumption changed relative to $(0.5, 0.25, 0.25)$?

Exercise 2. Replace the original meaning's two 0.25 aliases with five aliases of mass 0.1 each. Keep the other two class masses at 0.25. What happens to meaning entropy?

Exercise 3. A uniform trace universe contains 1,024 behaviors. Requirement A leaves 512; adding B leaves 128. Find $I(A)$, $I(B | A)$, and the remaining behavioral entropy.

Exercise 4. A reliable yes/no question isolates a meaning with probability 0.2 from the remaining mass 0.8. What is its EIG? Assume each meaning determines the answer.

Exercise 5. Why can all three experimental meanings collapse into one class at horizon two? Does zero bounded meaning entropy close the engineering decision?

Solutions.

1. $H = 0.921928$ bits. The candidate probabilities changed; this is a different prior or posterior, not new formal semantics.
2. Meaning entropy remains 1.5 bits: the original class still has mass 0.5. Label entropy becomes $-5(0.1 \log_2 0.1) - 2(0.25 \log_2 0.25) = 2.660964$ bits.
3. $I(A) = 1$ bit; $I(B | A) = 2$ bits; remaining entropy $\log_2 128 = 7$ bits.
4. $EIG = H(0.2, 0.8) = 0.721928$ bits.
5. No deadline of two or three ticks can expire within ticks 0–1. The equivalence is a horizon artifact. Candidate completeness and acceptance remain unassessed.

Research foundations. The implementation applies established concepts; entropy, model counting, and active learning are not claimed as new inventions.

1. FRETish semantics: *A Compositional Proof Framework for FRETish Requirements*. <https://arxiv.org/abs/2201.03641>
2. Grenn, Sarkani, and Mazzuchi: *The Requirements Entropy Framework in Systems Engineering* (2014). <https://doi.org/10.1111/sys.21283>
3. Kuhn, Gal, and Farquhar: *Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation* (2023). <https://arxiv.org/abs/2302.09664>
4. *Information-Guided Temporal Logic Inference with Prior Knowledge*. <https://arxiv.org/abs/1811.08846>

Repository and full methodological references:

https://github.com/volkanaranengineering/FRET_toy_project/tree/main/entropy